Progetto di Ingegneria del Software 2025/26

Università Ca' Foscari Venezia

D2: Piano di Progetto

Versione: 2.0

Gruppo: DSH

26/01/2026

Document Informations

AlertUp	AU
Deliverable	Piano di progetto
Data di Consegna	26/01/2026
Team Leader	Samuele Zanutto 890913@stud.unive.it
Team members	Davide Lago 895388@stud.unive.it Hamza Rofi 903405@stud.unive.it

Document History

Version	Issue Date	Stage	Changes	Contributors
1.0	21 / 10 / 2025	Draft	Definizione del piano di progetto.	Samuele Zanutto, Davide Lago, Hamza Rofi.
2.0	26/01/2026	Final	Allineamento documentazione: revisione del contenuto e correzioni dettagli.	Samuele Zanutto, Davide Lago, Hamza Rofi.

Indice

1. Introduzione 4
   1.1. Overview del progetto 4
   1.2. Deliverables 4
   1.3. Evoluzione del progetto 5
   1.4. Materiali di riferimento 5
   1.5. Definizioni e abbreviazioni 5
2. Organizzazione del progetto 7
   2.1. Modello del processo 7
   2.2. Struttura Organizzativa 7
   2.3. Interfacce Organizzative 8
   2.4. Responsabilità di progetto 9
3. Processi Gestionali 10
   3.1. Obiettivi e priorità 10
   3.2. Assunzioni, Dipendenze, Vincoli 10
   3.3. Gestione dei rischi 11
   3.4. Meccanismi di controllo 13
   3.5. Pianificazione dello staff 14
4. Processi Tecnici 14
   4.1. Metodi, Strumenti e Tecniche 14
   4.2. Documentazione 15
   4.3. Funzionalità di Supporto al progetto 16
5. Pianificazione delle attività e delle risorse 16
   5.1. Work Breakdown Structure 16
   5.2. Dipendenze 17
   5.3. Risorse necessarie 20
   5.4. Allocazione del budget e delle risorse 21
   5.5. Pianificazione 21
6. Materiali di Riferimento 22

1. Introduzione

1.1. Overview del progetto

L'obiettivo del progetto è lo sviluppo di un'applicazione Android per la gestione e segnalazione di emergenze climatiche, come alluvioni, incendi, neve, ghiaccio, temperature estreme o altro. L'app mira a fornire un sistema rapido e collaborativo di raccolta e diffusione delle informazioni, basato sulle segnalazioni dirette degli utenti.
La piattaforma consentirà di visualizzare su una mappa interattiva le emergenze segnalate, ricevere notifiche in tempo reale per eventi vicini e consultare un elenco aggiornato delle segnalazioni.
Particolare attenzione sarà riservata all'usabilità, con un'interfaccia semplice, leggibile e intuitiva, adatta anche a persone anziane o con ridotta familiarità tecnologica. L'obiettivo finale è migliorare la consapevolezza e la sicurezza della popolazione in caso di emergenze ambientali, favorendo la collaborazione tra cittadini e istituzioni.

1.2. Deliverables

Nome deliverable	Data consegna	Metodo di consegna
Definizione dei gruppi di lavoro	7/10/2025	Tramite Moodle del corso
Proposta iniziale	14/10/2025	Tramite Moodle del corso
Piano di Progetto	21/10/2025	Tramite Moodle del corso
Documento dei Requisiti	11/11/2025	Tramite Moodle del corso
Piano di Testing	18/11/2025	Tramite Moodle del corso
Documento di Progettazione	30/11/2025	Tramite Moodle del corso
Versione 1.0 del codice sorgente	22/12/2025	Tramite Moodle del corso
Versione 1.1. del codice e allineamento documentazione	26/01/2026	Tramite Moodle del corso

1.3. Evoluzione del progetto

Il progetto seguirà una crescita iterativa e incrementale. In una prima fase verrà realizzato un prototipo funzionale con interfaccia base, supporto alla registrazione utenti e sistema di segnalazione testuale. Le fasi successive prevedono:

- integrazione della mappa interattiva e geolocalizzazione;
- gestione delle immagini nelle segnalazioni;
- introduzione di notifiche push localizzate;
- ottimizzazione dell'interfaccia per accessibilità e usabilità;
- test ed eventuale debugging.

1.4. Materiali di riferimento

- Slides del corso di ingegneria del software
- Android Studio
- Android API documentation
- Angular
- MongoDB

1.5. Definizioni e abbreviazioni

• **App** – Applicazione software per dispositivi mobili. Fonte: https://developer.android.com/guide
• **Agile** – Metodologia di sviluppo iterativa e incrementale che privilegia la flessibilità e la collaborazione. (Fonte: https://agilemanifesto.org)
• **Sprint** – Iterazione di sviluppo a durata fissa tipica delle metodologie Agile. Fonte: https://scrumguides.org
• **Scrum** – Framework Agile per la gestione di progetti complessi. Fonte: https://scrumguides.org
• **Deliverable** – Risultato tangibile di una fase di progetto, come codice o documentazione. Fonte: https://www.pmi.org/learning/library/project-deliverables-importance-9709
• **Milestone (M)** – Traguardo significativo che segna il completamento di una fase. Fonte: https://www.pmi.org
• **WBS (Work Breakdown Structure)** – Scomposizione gerarchica del lavoro di progetto. Fonte: https://www.pmi.org
• **PERT (Program Evaluation and Review Technique)** – Metodo per analizzare le dipendenze e stimare la durata di un progetto. Fonte: https://asq.org/quality-resources/pert
• **Gantt** – Diagramma per rappresentare la pianificazione temporale delle attività. Fonte: https://www.gantt.com
• **Frontend** – Parte dell'applicazione che interagisce con l'utente. Fonte: https://developer.mozilla.org/en-US/docs/Learn/Front-end_web_developer
• **Backend** – Parte server dell'applicazione che gestisce logica e dati. Fonte: https://developer.mozilla.org/en-US/docs/Learn/Server-side
• **API REST** – Interfaccia per la comunicazione tra sistemi basata su HTTP e architettura REST. Fonte: https://restfulapi.net
• **Node.js** – Runtime JavaScript lato server. Fonte: https://nodejs.org
• **Angular** – Framework frontend sviluppato da Google per applicazioni web dinamiche. Fonte: https://angular.io
• **MongoDB** – Database NoSQL orientato ai documenti. Fonte: https://www.mongodb.com
• **JSON (JavaScript Object Notation)** – Formato standard di interscambio dati. Fonte: https://www.json.org
• **GitHub** – Piattaforma per versionamento e collaborazione sul codice sorgente. Fonte: https://github.com

• **Branch** – Linea di sviluppo parallela in un repository Git. Fonte: https://git-scm.com/docs/git-branch
• **Commit** – Salvataggio di una modifica nel repository. Fonte: https://git-scm.com/docs/git-commit
• **Pull request** – Revisione del codice per l'unione di branch. Fonte: https://docs.github.com/en/pull-requests
• **Main branch** – Branch principale del repository contenente la versione stabile del codice. Fonte: https://docs.github.com/en/get-started/quickstart/github-flow
• **Feature branch** – Branch dedicato allo sviluppo di una specifica funzionalità. Fonte: https://nvie.com/posts/a-successful-git-branching-model
• **UI/UX (User Interface / User Experience)** – Progettazione grafica e funzionale dell'esperienza utente. Fonte: https://www.interaction-design.org/literature/topics/ux-design
• **Material Design** – Linee guida grafiche di Google per interfacce coerenti. Fonte: https://m3.material.io
• **Testing** – Attività di verifica del corretto funzionamento del software. Fonte: https://www.iso.org/standard/45147.html
• **Unit test** – Test su singoli moduli o componenti software. Fonte: https://martinfowler.com/bliki/UnitTest.html
• **Integration test** – Test di interazione tra componenti integrate. Fonte: https://martinfowler.com/articles/practical-test-pyramid.html
• **QA (Quality Assurance)** – Controllo e verifica della qualità del prodotto. Fonte: https://asq.org/quality-resources/quality-assurance
• **Repository** – Archivio di codice versionato. Fonte: https://git-scm.com/book/en/v2/Git-Basics-Recording-Changes-to-the-Repository
• **Issue tracking** – Sistema di gestione dei problemi, bug e richieste. Fonte: https://docs.github.com/en/issues
• **Moodle** – Piattaforma di e-learning per consegna e gestione corsi universitari. Fonte: https://moodle.org
• **Configuration Management (CM)** – Gestione delle versioni e configurazioni software. Fonte: https://www.iso.org/standard/74808.html
• **Quality Control (QC)** – Processo di verifica della conformità agli standard. Fonte: https://asq.org/quality-resources/quality-control
• **Deployment** – Fase di rilascio e pubblicazione del software in ambiente operativo. Fonte: https://martinfowler.com/bliki/ContinuousDelivery.html
• **Android Studio** – IDE ufficiale per lo sviluppo Android. Fonte: https://developer.android.com/studio
• **NoSQL** – Modello di database non relazionale basato su documenti, chiave-valore o grafi. Fonte: https://www.mongodb.com/nosql-explained
• **RxJS** – Libreria JavaScript per la programmazione reattiva in Angular. Fonte: https://rxjs.dev
• **Visual Studio Code (VS Code)** – Editor di codice open source di Microsoft. Fonte: https://code.visualstudio.com
• **Discord / Google Meet** – Strumenti di comunicazione usati per meeting e coordinamento. Fonti: https://discord.com / https://meet.google.com

2. Organizzazione del progetto

2.1. Modello del processo

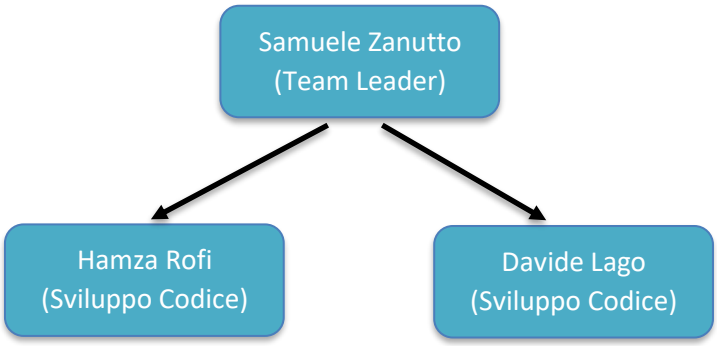
Il progetto adotta un modello di sviluppo Agile, in particolare una versione semplificata del framework Scrum, adattata alle dimensioni ridotte del team.

La scelta di un modello Agile è motivata dalla natura iterativa del progetto e dalla necessità di integrare progressivamente nuove funzionalità (es. segnalazioni, mappe, notifiche). Il modello consente di gestire in modo flessibile i requisiti, che possono evolvere sulla base dei feedback ricevuti durante lo sviluppo e dei test sui prototipi.

Ogni sprint prevede obiettivi chiari (implementazione di moduli specifici), una revisione intermedia e un'integrazione continua tra le componenti sviluppate. Tale approccio garantisce una consegna incrementale dei deliverables, una maggiore qualità del prodotto e una comunicazione costante tra i membri del gruppo.

2.2. Struttura Organizzativa

L'organizzazione del team segue una struttura funzionale leggera, in cui i ruoli sono distinti ma strettamente interconnessi.



L'organizzazione prevede un canale di comunicazione diretto e continuo tra i membri, e con brevi meeting di aggiornamento interni anche in videochiamata o in presenza ove possibile, per la gestione delle attività e l'assegnazione dei compiti, oltre che per la risoluzione di problematiche nel caso di difficoltà del singolo.

2.3. Interfacce Organizzative

● **Consultazione Progetto:** Il Team Leader, si interfaccerà con il professore A. Cortesi per quanto riguarda il monitoraggio e la valutazione dell'avanzamento del progetto. Da questi incontri sono attesi riscontri sulla qualità, completezza e coerenza dei deliverable. In caso di feedback negativo, il gruppo provvederà alla correzione e al miglioramento della parte interessata; in caso di feedback positivo, l'attenzione si sposterà sull'implementazione delle funzionalità successive e sulla preparazione dei deliverable futuri. Le consultazioni dovranno essere periodiche, così da garantire un controllo continuo sull'andamento del progetto. I canali di comunicazione utilizzati saranno la mail universitaria e i colloqui in presenza al termine delle lezioni o durante i ricevimenti del docente.

● **Consultazione Software:** Il Responsabile tecnico, in questo caso condiviso tra Hamza Rofi e Davide Lago, si interfaccerà con il professore A. Spanò per le attività di consulenza tecnica relative allo sviluppo software. Da queste interazioni sono attesi chiarimenti su problematiche legate alle tecnologie adottate (Angular, Android Studio, MongoDB) e alle scelte architetturali del progetto. Le comunicazioni avverranno tramite mail universitaria o incontri diretti durante le lezioni di laboratorio, con l'obiettivo di garantire il corretto allineamento tecnico alle linee guida del corso e alle buone pratiche di sviluppo.

● **Consultazione Esperienza Utente:**

Il Team Leader, in collaborazione con gli sviluppatori, si interfaccerà con i tester esterni incaricati di fornire feedback sull'esperienza d'uso dell'applicazione. Le consultazioni riguarderanno l'usabilità, la chiarezza dell'interfaccia e l'accessibilità per utenti con differenti competenze tecnologiche, in particolare anziani. Da questi confronti sono attesi suggerimenti per il miglioramento delle funzionalità, ottimizzazione della navigazione e segnalazione di bug o anomalie. Le modalità di comunicazione potranno variare a seconda dei tester, includendo incontri diretti, moduli di feedback digitali e sessioni di prova supervisionate.

Questo confronto continuo è essenziale per assicurare che il prodotto finale risulti utile, intuitivo e coerente con le esigenze reali degli utenti finali.

2.4. Responsabilità di progetto

Ruolo	Membro	Responsabilità
Project Manager e Documentation Supervisor	Samuele Zanutto	• Coordinamento generale del progetto. • Pianificazione attività e monitoraggio avanzamento. • Gestione e aggiornamento della documentazione tecnica e gestionale. • Responsabile dei deliverables e delle comunicazioni con il professore.
Sviluppatore lato Backend	Hamza Rofi	• Implementazione del backend (API REST) e connessione con MongoDB. • Gestione della logica di autenticazione, segnalazioni e mappe. • Supporto alla gestione delle immagini e all'interfaccia dati-frontend.
Sviluppatore lato Frontend	Davide Lago	• Implementazione dell'interfaccia utente in Angular. • Conversione e integrazione del progetto in Android Studio. • Supporto ai test di usabilità e ottimizzazione dell'interfaccia.

Tutti i membri partecipano alle fasi di revisione, test funzionali e validazione dei deliverables intermedi, con riunioni periodiche per la valutazione dello stato del progetto.

3. Processi Gestionali

3.1. Obiettivi e priorità

Gli obiettivi principali del progetto consistono nello sviluppo di un'applicazione Android stabile, funzionale ed efficiente, che consenta una gestione collaborativa delle emergenze climatiche. Le scadenze dei deliverable sono fisse, come stabilito dal piano di progetto e dai vincoli accademici. Tuttavia, l'organizzazione interna delle attività può essere modulata in funzione delle esigenze di sviluppo e dei risultati delle revisioni intermedie.

Il team privilegia l'efficienza rispetto alla qualità estetica, concentrandosi sull'affidabilità delle funzioni principali e sulla rapidità di completamento dei moduli essenziali. L'attenzione alla qualità verrà riservata principalmente all'usabilità dell'interfaccia e alla stabilità complessiva del software, senza compromettere i tempi di consegna.

3.2. Assunzioni, Dipendenze, Vincoli

Assunzioni:

- Tutti i membri del team dispongono di competenze di base in programmazione web e mobile.
- Gli strumenti di sviluppo (Angular, Android Studio, MongoDB) resteranno stabili durante il ciclo di progetto.
- Le comunicazioni con i docenti saranno regolari e garantite per la revisione dei deliverable.

Dipendenze:

- Il corretto funzionamento dell'app dipende dall'integrazione stabile tra frontend (Angular) e backend (MongoDB).
- Le tempistiche del progetto dipendono dal calendario accademico e dalle scadenze ufficiali dei deliverable.

Vincoli:

- Rispetto delle scadenze dei deliverable e le indicazioni fornite del docente.
- La realizzazione di un'applicazione intuitiva ed affidabile
- Rispetto della suddivisione del lavoro e della pianificazione decisa

3.3. Gestione dei rischi

Vengono elencati i principali rischi individuati durante il ciclo di vita del progetto, seguiti dalla relativa tabella di analisi. L'elenco è in ordine alfabetico e comprende rischi di natura organizzativa, tecnica e gestionale.

- **Abbandono di un membro del gruppo** – Uno dei tre componenti potrebbe interrompere la partecipazione, causando un aumento del carico di lavoro per gli altri membri.
- **Conflitti interni** – Divergenze su scelte tecniche o organizzative potrebbero compromettere l'efficienza del gruppo.
- **Guasti hardware** – Malfunzionamenti dei dispositivi utilizzati (PC, smartphone per test) possono generare ritardi.
- **Inadeguata pianificazione dei task** – La scarsa esperienza nella gestione di progetti complessi può portare a una distribuzione inefficiente delle attività.
- **Inesperienza tecnica** – La limitata conoscenza di Angular, MongoDB e Android Studio può rallentare lo sviluppo iniziale.
- **Interruzione o malfunzionamento del servizio cloud** – Problemi con MongoDB ho con la connessione di rete potrebbero causare perdita temporanea di accesso ai dati.
- **Perdita o corruzione dei dati** – Errori di sincronizzazione, salvataggio o versionamento potrebbero comportare la perdita di parti di codice o documentazione.
- **Sovrapposizione con impegni accademici o personali** – Esami, lezioni o impegni esterni potrebbero ridurre il tempo disponibile per il progetto.
- **Ritardi nella comunicazione interna** – Una comunicazione non tempestiva tra i membri può generare incoerenze e rallentamenti.
- **Ritardi nelle consegne intermedie** – Eventuali difficoltà tecniche o organizzative possono compromettere il rispetto delle scadenze stabilite.

ID	Rischio	Probabilità	Impatto	Azioni di mitigazione	Monitoraggio
1	Abbandono di un membro del gruppo	1	4	Suddivisione chiara delle responsabilità e repository condiviso per garantire la continuità	Verifica settimanale dello stato del lavoro
2	Conflitti interni	2	2	Riunioni di coordinamento e revisione delle decisioni tramite il team leader	Controllo del clima collaborativo nelle review
3	Guasti hardware	1	3	Backup periodici e utilizzo di repository Git	Verifica dell'integrità dei dati a ogni push
4	Inadeguata pianificazione dei task	2	3	Continua comunicazione e aggiornamento sull'avanzamento di ogni task	Analisi delle milestone completate
5	Inesperienza tecnica	3	3	Formazione autonoma e divisione del lavoro in base alle competenze	Monitoraggio della curva di apprendimento
6	Malfunzionamento del servizio cloud	2	4	Backup locale dei dati e piani alternativi di connessione	Verifica periodica della disponibilità del servizio
7	Perdita o corruzione dei dati	2	4	Uso costante di Git e salvataggi su repository remoto	Controllo settimanale della coerenza dei file
8	Sovrapposizione con impegni accademici	3	2	Pianificazione flessibile e suddivisione del carico in anticipo	Revisione del calendario in base alle sessioni d'esame
9	Ritardi nella comunicazione interna	2	1	Utilizzo di canali unificati (es. gruppo WhatsApp, GitHub)	Verifica della tempestività nelle risposte
10	Ritardi nelle consegne intermedie	2	4	Anticipare il completamento dei task critici e mantenere buffer temporali	Confronto settimanale sui progressi effettivi

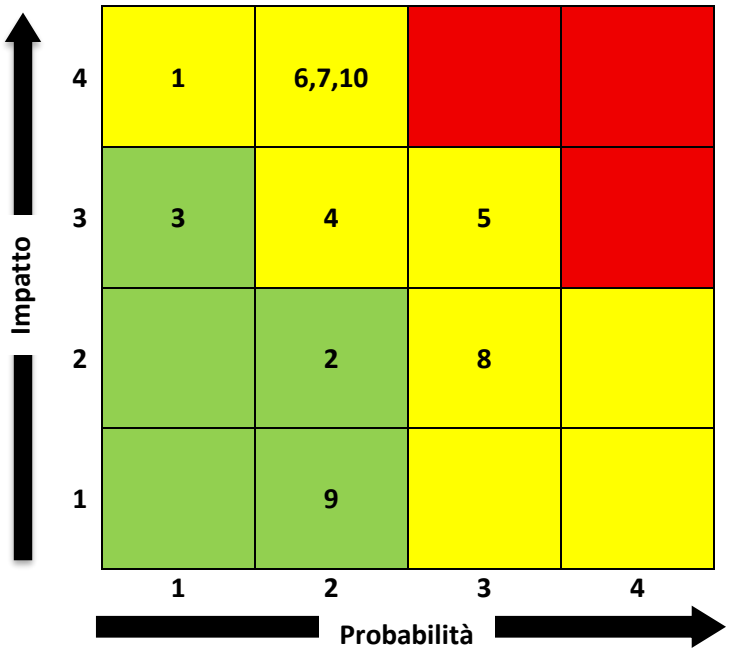
Probabilità:

1. Poco probabile
2. Probabile
3. Altamente probabile
4. Quasi certo

Impatto:

1. Lieve
2. Moderato
3. Significativo
4. Grave

Modello dei rischi:



3.4. Meccanismi di controllo

Il team adotta una gestione agile dei progressi tramite aggiornamento quotidiano, in cui ogni membro informa dei progressi fatti e di eventuali difficoltà riscontrate. Il Team Leader supervisiona l'andamento complessivo, aggiornando lo stato di avanzamento e segnalando eventuali scostamenti rispetto al piano previsto.

La comunicazione interna avviene tramite canali digitali condivisi (es. gruppo WhatsApp e repository GitHub), garantendo tracciabilità e trasparenza.

Al termine di ogni sprint o fase significativa, verrà redatto un report sintetico contenente: attività completate, criticità riscontrate, azioni correttive pianificate e stato di avanzamento rispetto alle milestone del progetto.

3.5. Pianificazione dello staff

- **Competenze presenti:**
  - Samuele Zanutto: coordinamento, documentazione, gestione deliverable.
  - Davide Lago: sviluppo frontend, integrazione Angular–Android Studio.
  - Hamza Rofi: sviluppo backend, gestione database MongoDB.
- **Competenze richieste:**
  - Gestione di notifiche push e geolocalizzazione in Android.
  - Ottimizzazione delle prestazioni e affidabilità applicazione.
- **Formazione pianificata:**
  - Autoformazione tramite documentazione ufficiale Angular e MongoDB.
  - Esercitazioni pratiche su progetti di prova per consolidare l'integrazione Angular–Android Studio.
  - Condivisione di conoscenze tecniche tra i membri mediante brevi sessioni interne di aggiornamento.
  - Sfruttamento delle lezioni fornite dal professor A. Spanò ed eventuale richiesta di ricevimento per consigli pratici.

Il team adotta un approccio di apprendimento continuo, con scambio costante di competenze e supporto reciproco per garantire la copertura di tutte le aree tecniche necessarie allo sviluppo completo dell'applicazione.

4. Processi Tecnici

4.1. Metodi, Strumenti e Tecniche

Ambiente di sviluppo

Lo sviluppo dell'app avviene in un ambiente **ibrido**, basato su **Angular** per la parte frontend e **Android Studio** per la fase di conversione e distribuzione mobile.
Il backend utilizza **MongoDB**, un database NoSQL che consente una gestione flessibile dei dati, adatta a un'applicazione che integra informazioni in tempo reale e segnalazioni utente.
L'ambiente di sviluppo è organizzato come segue:

- **Frontend**: Angular (JavaScript, HTML, CSS, TypeScript).
- **Backend**: Node.js con interfacciamento a MongoDB.
- **Ambiente mobile**: Android Studio per il packaging, testing e pubblicazione dell'app su dispositivi Android.
- **Controllo versione**: GitHub per la gestione del codice e delle revisioni.

Motivazione della scelta di Angular

La scelta di Angular è motivata dalla necessità di un **framework strutturato e modulare**, adatto a gestire un'interfaccia dinamica e una logica di aggiornamento in tempo reale.
Vantaggi principali:

- Architettura **MVC (Model-View-Controller)** che facilita la separazione tra logica e interfaccia.
- Ampio supporto a **componenti riutilizzabili**, che semplificano l'estensione futura del progetto.
- **Compatibilità con API RESTful**, che consente una facile integrazione con il backend MongoDB.
- **Gestione reattiva** tramite RxJS, utile per aggiornamenti continui delle segnalazioni e dei dati ambientali.

- Ecosistema consolidato e forte supporto dalla community.

Svantaggi principali:

- Curva di apprendimento relativamente **ripida**, che richiede familiarità con TypeScript e con la gestione dei moduli.
- **Overhead iniziale** nella configurazione del progetto rispetto a framework più leggeri (es. Vue.js).
- Necessità di un'accurata **ottimizzazione delle performance** per garantire fluidità su dispositivi mobili con risorse limitate.

Linguaggi utilizzati

- **JavaScript / TypeScript** per la logica e la gestione dei componenti frontend.
- **HTML / CSS** per la struttura e la presentazione grafica.
- **Node.js** per la gestione del backend e l'interfacciamento con MongoDB.
- **JSON** come formato di scambio dati tra frontend e backend.

Standards e Linee Guida

Il progetto segue linee guida comuni di sviluppo e documentazione:

- Convenzioni di naming coerenti (camelCase per variabili e funzioni, PascalCase per componenti).
- Strutturazione modulare del codice secondo le **Angular Best Practices**.
- Commenti in linea e file README.md aggiornati per ciascun modulo.
- Conformità agli standard **RESTful API** per la comunicazione client-server.
- Rispetto delle linee guida **Material Design** per l'interfaccia utente.

4.2. Documentazione

La documentazione di progetto è suddivisa in due tipologie principali:

1. **Documentazione gestionale**, che include:
    - Piano di progetto e deliverables.
    - Report periodici di avanzamento.
    - Revisioni interne approvate dal team prima della consegna al docente.

    Ogni documento viene **revisionato a conclusione di ogni sprint interno** e validato collettivamente.

2. **Documentazione tecnica**, integrata nel codice sorgente:
    - Commenti inline e descrittivi di classi, funzioni e moduli.
    - File README.md per ogni sezione del progetto (frontend, backend, database).
    - Schemi ER e diagrammi architetturali aggiornati in base all'evoluzione del codice.

    La documentazione del codice è revisionata **a ogni commit significativo** e verificata in sede di merge su GitHub.

4.3. Funzionalità di Supporto al progetto

Pianificazione della qualità

La qualità del progetto è monitorata costantemente dal *Team Leader*, che verifica la coerenza tra requisiti e implementazione.

Ogni nuova funzionalità passa attraverso:

- **Code review interna** da parte di almeno un membro del team.
- **Testing manuale** delle interfacce e dei flussi principali.
- **Verifica formale** del rispetto delle linee guida di stile e struttura.
  I controlli qualitativi vengono effettuati **al termine di ogni milestone** del progetto (fine modulo o nuova funzionalità completata).

Gestione delle configurazioni

La gestione delle versioni e delle configurazioni è affidata a **GitHub**, con la seguente struttura organizzativa:

- Branch principale main: versione stabile del codice.
- Branch di sviluppo dev: ambiente di integrazione delle nuove feature.
- Branch dedicati per funzionalità (feature/<nome>).
- Utilizzo di **commit descrittivi e numerati**, in modo da garantire tracciabilità e rollback.
- Creazione di **release taggate** in corrispondenza dei deliverables approvati.

Tutti i membri del gruppo contribuiscono al repository, seguendo una policy di **pull request obbligatoria** per evitare conflitti e garantire la validazione del codice prima dell'integrazione.

5. Pianificazione delle attività e delle risorse

5.1. Work Breakdown Structure

Il progetto adotta un **modello Agile**, con sviluppo iterativo e rilascio progressivo delle funzionalità. La pianificazione è organizzata in **sprint** che seguono le scadenze dei deliverable ufficiali. Le funzioni e attività qui riportate rappresentano la scomposizione logica del lavoro fino al completamento della versione 1.1. dell'applicazione.

FUNZIONI

- **F1 – Project Management:** pianificazione sprint, meeting settimanali, monitoraggio avanzamento, gestione rischi e consegne.
- **F2 – Configuration Management (CM):** gestione repository GitHub, branch policy, backup e ambiente di sviluppo.
- **F3 – Documentation:** redazione e aggiornamento Proposta iniziale, Piano di progetto, Documento dei requisiti, Piano di testing, Documento di progettazione, manuali.
- **F4 – Quality Control:** review di codice, verifica requisiti e controlli di conformità ad ogni sprint.
- **F5 – Development:** sviluppo frontend (Angular → packaging Android) e backend (Node.js + MongoDB).
- **F6 – Testing:** test unitari, integrazione, usabilità con tester esterni.
- **F7 – Deployment & Release:** packaging Android, rilascio Versione 1.0 e Versione 1.1, allineamento documentazione.

ATTIVITÀ

- **A1** Definizione gruppo e assegnazione ruoli
- **A2** Raccolta idee e selezione soluzione
- **A3** Definizione obiettivi e requisiti preliminari
- **A4** Redazione Proposta iniziale
- **A5** Redazione Piano di Progetto
- **A6** Analisi dettagliata e modellazione requisiti
- **A7** Progettazione architetturale
    - **A7.1** Schema logico database
    - **A7.2** Definizione API REST (→ Documento dei Requisiti / M1)
- **A8** Implementazione backend & setup
    - **A8.1** Implementazione API e connessione DB + Piano di testing (→ Piano di Testing / M2)
    - **A8.2** Testing / verifiche backend
    - **A8.3** Documento di Progettazione
- **A9** Sviluppo frontend e integrazione
    - **A9.1** Componenti e schermate base
    - **A9.2** Integrazione con API e features (mappe, segnalazioni, immagini)
    - **A9.3** Ottimizzazione UI/UX e accessibilità
- **A10** Preparazione Versione 1.0 (build, packaging, test finali) → Versione 1.0 (22/12/2025)
- **A11** Messa a punto per Versione 1.1 e allineamento documentazione → Consegna finale (26/01/2026)

MILESTONES

- **M1:** Documento dei Requisiti (11/11/2025)
- **M2:** Piano di Testing (18/11/2025) / Documento di Progettazione (30/11/2025)
- **M3:** Versione 1.0 del codice sorgente (22/12/2025)
- **M4:** Versione 1.1 e allineamento documentazione (26/01/2026)

5.2. Dipendenze

Questa sezione presenta le relazioni di precedenza tra le attività e i task individuati nella **WBS** del progetto. L'obiettivo è definire una pianificazione coerente con il modello **Agile**, garantendo flessibilità nelle iterazioni ma mantenendo un ordine logico tra le attività fondamentali. La definizione delle dipendenze consente di individuare attività sequenziali, parallele e critiche, facilitando la gestione dei tempi e la distribuzione del carico di lavoro.
La stima delle durate e delle relazioni tra le attività è fondamentale per monitorare l'avanzamento del progetto e prevenire ritardi nella consegna delle milestone principali. Alcune attività possono essere svolte in parallelo (ad esempio lo sviluppo del backend e la progettazione grafica dell'app), mentre altre dipendono strettamente dal completamento di fasi precedenti (come l'integrazione delle API, che richiede la definizione del database).

Definizione delle variabili temporali

- **ES (Earliest Start)**: giorno più precoce in cui un'attività può iniziare, in base al completamento delle attività precedenti.
- **EF (Earliest Finish)**: giorno più precoce in cui l'attività può terminare, dato ES e la durata prevista.
- **LS (Latest Start)**: ultimo giorno utile per iniziare l'attività senza causare ritardi nelle successive.
- **LF (Latest Finish)**: ultimo giorno utile per completare l'attività senza influenzare negativamente il progetto.

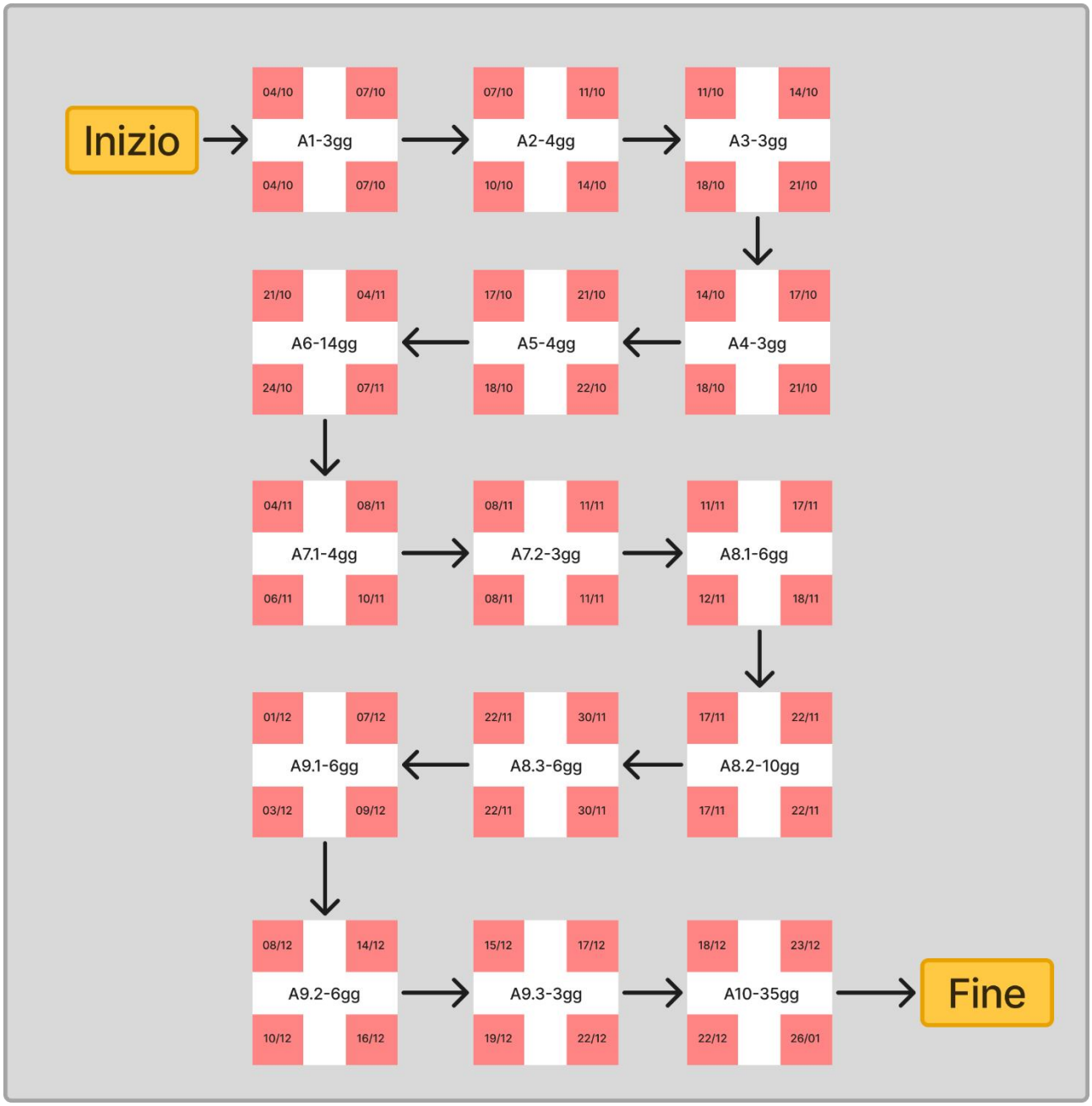
Tabella delle Dipendenze

ID	Dipendenze	Durata (gg)	ES	EF	LS	LF
A1	-	3	04/10/2025	07/10/2025	04/10/2025	07/10/2025 (Definizione gruppi)
A2	A1	4	07/10/2025	11/10/2025	10/10/2025	14/10/2025 (Proposta iniziale)
A3	A2	3	11/10/2025	14/10/2025	18/10/2025	21/10/2025 (Piano di Progetto)
A4	A3	3	14/10/2025	17/10/2025	18/10/2025	21/10/2025
A5	A4	4	17/10/2025	21/10/2025	18/10/2025	22/10/2025
A6	A5	14	21/10/2025	04/11/2025	24/10/2025	07/11/2025
A7	A6	7	04/11/2025	11/11/2025	04/11/2025	11/11/2025 (M1: Documento Requisiti)
A7.1	A7	4	04/11/2025	08/11/2025	06/11/2025	10/11/2025
A7.2	A7.1	3	08/11/2025	11/11/2025	08/11/2025	11/11/2025 (M1: Documento Requisiti)
A8	A7.2	19	11/11/2025	30/11/2025	11/11/2025	30/11/2025 (Documento di Progettazione, M2)
A8.1	A8	6	11/11/2025	17/11/2025	12/11/2025	18/11/2025(M2: Piano di Testing)
A8.2	A8.1	5	17/11/2025	22/11/2025	17/11/2025	22/11/2025
A8.3	A8.2	8	22/11/2025	30/11/2025	22/11/2025	30/11/2025 (Documento di Progettazione)
A9	A8.3	15	01/12/2025	19/12/2025	07/12/2025	22/12/2025 (M3: Versione 1.0)
A9.1	A9	6	01/12/2025	07/12/2025	03/12/2025	09/12/2025
A9.2	A9.1	6	08/12/2025	14/12/2025	10/12/2025	16/12/2025
A9.3	A9.2	3	15/12/2025	17/12/2025	19/12/2025	22/12/2025
A10	A9.3	35	18/12/2025	23/01/2026	22/12/2025	26/01/2026 (M4: Versione 1.1)

Note:

- Le attività A1–A5 coincidono con le prime fasi di ideazione e pianificazione.
- A6–A8 coprono l'analisi, progettazione e implementazione (deliverable: Documento dei Requisiti e Documento di Progettazione).
- A9–A11 rappresentano la fase di sviluppo, testing e rilascio finale.
- Le milestone M1, M2, M3 corrispondono ai punti chiave allineati ai deliverable.

Diagramma di Pert:



5.3. Risorse necessarie

Stima del tempo e pianificazione delle attività:

L'impegno medio stimato per completare il progetto è di circa **85 giorni lavorativi effettivi**, distribuiti su quattro mesi (ottobre 2025 – gennaio 2026).

Il team è composto da 3 membri, ciascuno con competenze differenziate ma parzialmente sovrapponibili per garantire flessibilità in ambiente Agile.

Pianificazione in base alle competenze (skills mapping)

Ruolo	Competenze principali	Attività assegnate / prevalenti	Carico stimato
Project Manager	coordinamento, pianificazione, risk management, documentazione	F1, F3, F4, A1–A5, revisione finale deliverable	~25%
Backend Developer	Node.js, MongoDB, REST API, testing	F5, F6, A8, A9.2	~30%
Frontend Developer	Angular, UI/UX, responsive design, Android packaging	F5, F7, A9.1–A9.3, A10	~30%
QA & Documentation	testing, validazione, documentazione tecnica	F4, F6, A8.2, A10–A11	~15%

Strumenti principali

- **Ambiente di sviluppo:** Visual Studio Code, Android Studio, MongoDB Compass
- **Versionamento:** GitHub (gestione branch, merge request, backup)
- **Gestione progetto:** GitHub Projects per task tracking Agile
- **Comunicazione interna:** Discord e Google Meet per riunioni di sprint review
- **Risorse hardware:** includono i dispositivi necessari per la programmazione, il test e la validazione:
  - PC/Laptop personali dei membri del team
  - Connessione di rete stabile.
- **Risorse temporali:** corrispondono al totale delle ore necessarie per completare l'intero ciclo di sviluppo.
  - Ore per persona = 17 settimane × 40 h/week = **680 h**
  - Ore totali team = 680 h × 3 = **2040 h**

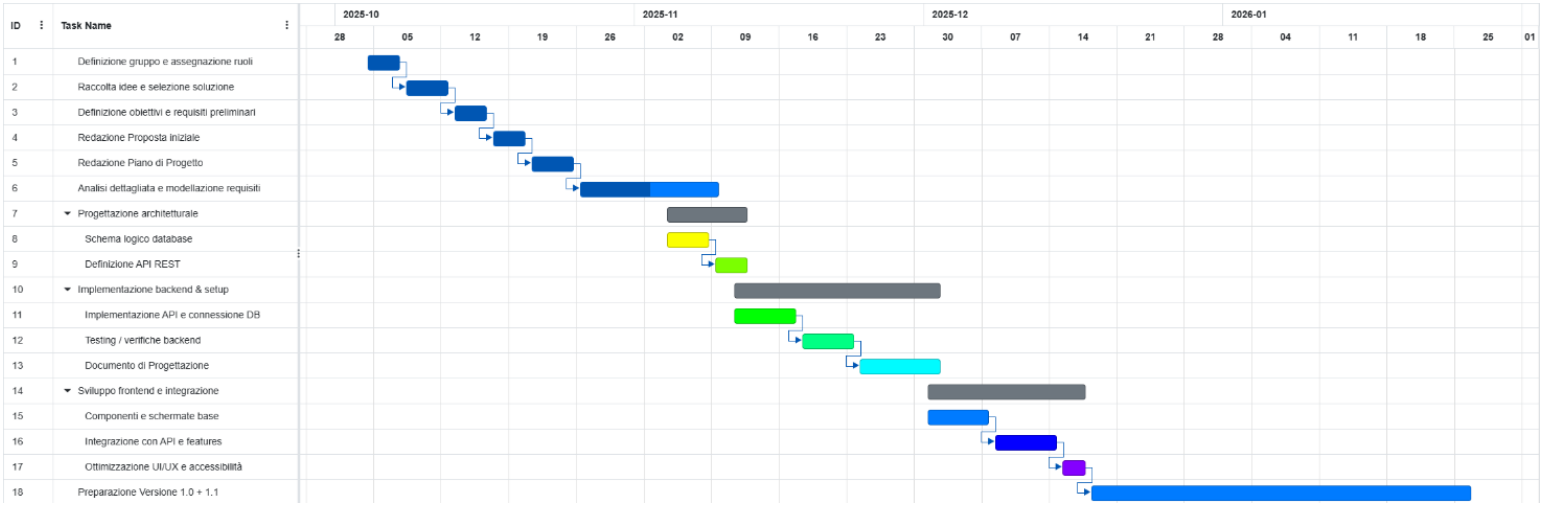
5.4. Allocazione del budget e delle risorse

I costi complessivi del progetto sono stati stimati tenendo conto delle ore di lavoro dei membri del team, delle risorse software impiegate, delle infrastrutture di rete e delle attività di documentazione.

Gruppo	Settore	Costo stimato (€)	Totale Gruppo
Ore di lavoro	3 membri × 680 ore × 15 €/h	—	**30.600 €**
Documentazione	Piano di progetto	150	**600 €**
	Documento dei requisiti	200	
	Piano di testing	150	
	Documento di progettazione	100	
Backend	Configurazione e gestione MongoDB	100	**300 €**
	API REST e sicurezza backend	150	
	Testing e debugging	50	
Frontend	Implementazione interfaccia Angular	250	**450 €**
	Integrazione con Android Studio	150	
	Testing e ottimizzazione UI	50	
Totale complessivo stimato	—	—	**36.000 €**

5.5. Pianificazione

Viene inserito di seguito il diagramma di Gantt che rappresenta la pianificazione delle attività.



6. Materiali di Riferimento

La presente sezione elenca i materiali, i documenti e le fonti di riferimento utilizzati per la redazione del **Piano di Progetto** e per la pianificazione tecnica e gestionale dello sviluppo dell'applicazione *AlertUp*. I materiali sono suddivisi in tre categorie principali: **documentazione interna**, **documentazione esterna** e **riferimenti tecnici e normativi**.

Materiali di supporto e studio utilizzati per la pianificazione del progetto e lo sviluppo dell'applicazione:

- **Slide e materiali del corso di Ingegneria del Software** – prof. *Agostino Cortesi*, Università Ca' Foscari Venezia.
  Contengono linee guida sui modelli di processo, gestione dei rischi, WBS, PERT, Gantt, testing e qualità del software.
- **Documentazione ufficiale Angular** – https://angular.io/docs
- **Documentazione ufficiale MongoDB** – https://www.mongodb.com/docs
- **Documentazione ufficiale Node.js** – https://nodejs.org/en/docs
- **Documentazione Git e GitHub** – https://git-scm.com/docs, https://docs.github.com
- **Material Design Guidelines (Google)** – https://m3.material.io
- **Scrum Guide (2020)** – https://scrumguides.org
- **Agile Manifesto (2001)** – https://agilemanifesto.org

Riferimenti tecnici e normativi

Documenti di standardizzazione e linee guida consultati a supporto delle scelte metodologiche e qualitative:

- **ISO/IEC 12207:2017** – *Systems and software engineering — Software life cycle processes*
- **ISO/IEC 25010:2011** – *System and software quality models*
- **IEEE Std 1058-2017** – *IEEE Standard for Software Project Management Plans*